

Darwoft

Take-Home Challenge

Analytics Engineer — Senior

Rol	Analytics Engineer (SQL / BI / dbt) · Senior
Empresa	Darwoft — producto fintech / servicios financieros

1. Contexto

Trabajás en el equipo de datos de un producto fintech de pagos digitales. El sistema procesa transacciones de usuarios en múltiples países y tipos de cuenta. Tu misión en este challenge es la misma que tendrías en el día a día:

transformar datos crudos en información accionable y construir los modelos que permitan a los equipos de negocio tomar mejores decisiones.

El challenge está dividido en tres partes independientes. Podés entregarlas en cualquier orden, aunque la Parte 1 (SQL) alimenta conceptualmente a la Parte 2 (dbt).

2. Dataset

Trabajás con tres tablas que representan la capa raw de un data warehouse:

Tabla	Filas aprox.	Descripción
users	500	Usuarios del producto. Campos: user_id, signup_date, country, plan_type (free / premium).
accounts	620	Cuentas asociadas a cada usuario. Campos: account_id, user_id, account_type (checking / savings), created_at.
transactions	~3.700	Transacciones. Campos: transaction_id, account_id, amount, currency (ARS / USD), transaction_type (deposit / withdrawal / payment / transfer), status (completed / failed / pending), created_at.

Relaciones: users → accounts (1:N) → transactions (1:N)

Entorno: las tablas ya están cargadas en DB Fiddle (MySQL 9). Ver instrucciones en el Anexo A.

3. Las tres partes del challenge

Parte 1 — SQL (MySQL 9)

Escribí las queries necesarias para responder las siguientes preguntas. Cada pregunta debe tener su propio archivo (ej. `q1.sql`, `q2.sql`, etc.).

Las respuestas tienen que devolver exactamente lo que se pide. Si la pregunta dice "un solo número", la query no debe devolver columnas extra.

Preguntas

Q1. ¿Cuántas transacciones completadas hubo por mes? Devolvé: mes (formato YYYY-MM), cantidad de transacciones y monto total (solo moneda ARS).

Q2. Para cada usuario, calculá el saldo neto acumulado considerando únicamente transacciones completadas. Devolvé: `user_id` y `saldo_net`. Ordená de mayor a menor saldo.

Q3. Mostrá, para cada cuenta, el saldo acumulado transacción por transacción (running balance) en orden cronológico. Incluí solo transacciones completadas.

Q4. ¿Qué porcentaje de transacciones falló, por tipo de transacción? Devolvé: `transaction_type`, total de transacciones, cantidad fallidas y tasa de fallo (como porcentaje redondeado a 2 decimales).

Q5. Identificá los usuarios que se registraron en enero 2024 y realizaron al menos una transacción completada en cada uno de los 2 meses siguientes a su signup. Devolvé sus `user_id`.

Parte 2 — Modelado con dbt

No es necesario tener un proyecto dbt corriendo. Entregá los archivos `.sql` y `.yaml` como si fueran parte de un proyecto dbt real, respetando las convenciones de estructura y sintaxis.

2.a — Modelo de staging

Creá el modelo `stg_transactions.sql` que:

- Lea desde `{{ source('raw', 'transactions') }}`
- Castee `amount` a `DECIMAL` y `created_at` a `TIMESTAMP`
- Excluya transacciones con `status = 'failed'`
- Agregue una columna calculada `is_credit` (TRUE si `amount > 0`, FALSE si `amount ≤ 0`)

2.b — Modelo mart

Creá el modelo `mart_user_activity.sql` que consolide, por usuario y mes:

- Cantidad de transacciones completadas
- Monto total de créditos (depósitos e ingresos)
- Monto total de débitos (pagos y retiros)
- Saldo neto del mes

El modelo debe referenciar `stg_transactions` usando `{{ ref() }}`.

2.c — Tests en YAML

Definí en un archivo `schema.yml` al menos:

- Un test de unicidad y uno de `not_null` sobre `stg_transactions`
- Un test de integridad referencial (`relationships`) que sea relevante
- Un test genérico adicional a tu elección, con un comentario que justifique la decisión

2.d — Pregunta conceptual (respuesta escrita, máx. 10 líneas)

Si la tabla de transacciones crece a 50M de filas por mes, ¿qué materialización de dbt elegirías para `stg_transactions` y para `mart_user_activity`? Justificá la decisión considerando costo de cómputo y frecuencia de actualización.

Parte 3 — Análisis e insights

Con los mismos datos, realizá un análisis exploratorio y presentá tus hallazgos. Podés usar la herramienta que prefieras: Power BI, Tableau, un notebook de Python/R, o incluso slides con capturas. Lo que importa es la calidad del razonamiento, no la herramienta.

Consigna

El equipo de Producto te pide entender el comportamiento transaccional de los usuarios para priorizar iniciativas del próximo trimestre. En particular quiere saber:

- ¿Existe algún período donde el volumen de transacciones haya caído de forma anómala? ¿A qué podría deberse?
- ¿Hay diferencias relevantes en el comportamiento entre usuarios free y premium que justifiquen una acción de producto?
- ¿Qué tipo de transacción tiene la mayor tasa de error y qué impacto tiene eso en el negocio?

Otros patrones descubiertos en los datos serán valorados.

Formato del entregable

- Máximo 1 página de texto + visualizaciones de soporte (no hay límite de gráficos)
- Si usás un dashboard, compartí el link o un PDF exportado
- Si usás un notebook, exportalo como HTML o PDF
- Documentá los supuestos que hagas sobre los datos

4. Criterios de evaluación

Evaluamos las tres partes con el mismo peso. Dentro de cada parte, priorizamos:

Criterio	Qué miramos
Correctitud	La query/modelo devuelve el resultado esperado.
Claridad del código	Código legible, bien indentado, con nombres descriptivos. En dbt: uso correcto de CTEs y convenciones de capas.
Razonamiento	Se entiende por qué tomaste cada decisión. En la parte conceptual de dbt y en el análisis, esto vale más que la respuesta "correcta".
Impacto de negocio	En la Parte 3: ¿los insights son accionables? ¿conectan los datos con decisiones reales de producto?
Manejo de edge cases	Usuarios sin transacciones, monedas mixtas, valores nulos: ¿los contemplás o los mencionás?

5. Entregables

Envía un ZIP o link a un repositorio (GitHub/GitLab) con la siguiente estructura:

```
challenge/
├── parte1/
│   ├── q1.sql
│   ├── q2.sql
│   ├── q3.sql
│   ├── q4.sql
│   └── q5.sql
├── parte2/
│   ├── models/
│   │   ├── staging/
│   │   │   └── stg_transactions.sql
│   │   └── marts/
│   │       └── mart_user_activity.sql
│   ├── schema.yml
│   └── respuesta_2d.md
└── parte3/
    └── [dashboard / notebook / slides]
```

Anexo A — Cómo usar DB Fiddle

Seguí estos pasos para tener el entorno listo en menos de un minuto:

1	Abrí dbfiddle.uk en tu navegador.
2	En el selector de motor (arriba a la izquierda) elegí MySQL 9.
3	Abrí el archivo dbfiddle_setup.sql que se adjunta junto a este documento.
4	Copió todo el contenido y pegalo en el panel izquierdo ("Schema SQL").
5	Hacé clic en Run. Las tres tablas quedan creadas y cargadas con todos los datos.
6	Escribí tus queries en el panel derecho ("Query SQL") y ejecutá con Run.

Nota: el archivo dbfiddle_setup.sql contiene el schema completo y todos los INSERT statements.